

Interview Q&A — built from your CV

27 questions your own resume lines invite · Md. Ziaur Rahman

How to use this. Every question is triggered by a specific line on your resume — printed above each question. Cover the answer, say yours out loud, then compare. The answers are shapes to speak in your own words, not scripts. One section a day. If an answer describes something you would not do or did not do, change it: an answer you cannot defend is worse than no answer.

Opening & screening

These come first in almost every interview, before anything technical. Get them boringly smooth.

FROM YOUR CV — Summary — '10 years... clients in Europe and Bangladesh'

01 Tell me about yourself.

WHY THEY ASK

They are checking whether you can be concise and whether your story has a shape. Rambling here damages everything after it.

YOUR ANSWER

I'm a full-stack engineer with ten years building production software — booking platforms, marketplaces, CRMs, and SaaS products. Most of my career has been PHP: Laravel and Symfony with Vue on the frontend, largely for international clients. For the past year I've been on remote contract with a Norwegian startup, where I owned an AI recruitment platform end to end — C# and ASP.NET Core on the backend, a Python service for the LLM scoring work, React and Next.js on the front, running on Azure — and shipped a lead-generation platform alongside it. I also build a small SaaS product of my own, which keeps me hands-on with multi-tenancy, offline sync and payments. I'm looking for a senior full-stack role where I own features end to end.

FOLLOW-UPS TO EXPECT

What are you looking for in your next role?

A senior full-stack role where I own features end to end rather than being handed tickets — ideally a product team where I can see how what I build affects the business. I work best with real ownership and written, async communication.

Why this company?

Prepare one specific, verifiable reason per company: their product domain overlaps something you've built, their stack matches yours, or something concrete from their engineering blog. Never a generic 'you're innovative'.

TRAP

Do not exceed 90 seconds and do not start at university. Start at 'I'm a full-stack engineer with ten years'.

FROM YOUR CV — Osilion — 'Sep 2025 — Sep 2026' (contract)

02 Why is the contract ending? Why are you leaving?

WHY THEY ASK

They are looking for a red flag — performance problems or a difficult exit. Your job is to close it calmly in two sentences.

YOUR ANSWER

It was a fixed-term contract with a startup. Their business situation changed and they couldn't sustain the development arrangement, so the contract ended as scheduled at the end of September. Both founders have offered to act as references.

FOLLOW-UPS TO EXPECT

Would you have stayed if they could have kept you?

Yes — I enjoyed the work and the products were interesting to build. It simply wasn't something either side could control.

How did you find the contract arrangement?

It suited that stage well and gave me a lot of ownership quickly. What I want now is more stability and a longer runway with one product, which is why I'm targeting permanent roles.

TRAP

Then stop talking. No bitterness, no long backstory, no criticism of the company. Offering references immediately converts this into a positive.

03 How do you handle the timezone difference?

WHY THEY ASK

This is the single biggest hesitation for a European employer hiring from Bangladesh. Answer it with evidence, not reassurance.

YOUR ANSWER

I've been doing exactly this for the past year with a Norwegian client, so it's a known quantity rather than a plan. Bangladesh is four to five hours ahead of Central Europe, which gives me a solid overlap through your morning and into the early afternoon. I work written-first — decisions in writing, status updates before anyone has to ask, and questions posted with enough context that the answer can come while I'm asleep and still be useful. I flag blockers within hours rather than sitting on them.

FOLLOW-UPS TO EXPECT

What hours can you commit to?

State a real window and hold it — e.g. 1pm-9pm Bangladesh time gives roughly 9am-5pm Central Europe. Commit to an overlap you can sustain for years, not a heroic one you'll resent in three months.

How did you handle urgent issues at Osilion?

Agreed escalation expectations up front: anything production-breaking I'd pick up outside the overlap window, everything else waited for the next working block. Good logging meant most issues were diagnosable without waking anyone.

TRAP

Never say 'I'm flexible, I can work any hours'. It sounds desperate and it isn't sustainable. State a real overlap window and hold it.

04 How do you use AI in your development work?

WHY THEY ASK

Now asked in most interviews. They want to know whether you understand what you ship. Your honest answer is a strength if framed correctly.

YOUR ANSWER

I use it as an accelerator, not a substitute for understanding. I design the architecture and the data model myself, then use AI to move faster on implementation — and I review everything that goes in, because I have to debug it at 2am and defend it in code review. On my own product I wrote the platform standards and the sync design first, then built against them. Where I don't let it near anything is security boundaries and data modelling; those I do by hand.

FOLLOW-UPS TO EXPECT

How much of your code is AI-written?

Varies by task — boilerplate and repetitive implementation, a lot; architecture, data modelling and anything security-related, none. The measure I hold myself to is whether I could rewrite it and explain every decision in review. If I can't, it doesn't ship.

Show me something you had to rewrite because AI got it wrong.

Have one real example ready. Common honest ones: a generated query that worked but triggered N+1 under load, an auth check that validated the token but never checked ownership, or a migration that wasn't backward-compatible.

TRAP

Never use the phrase 'vibe coded'. Never say 'AI wrote it'. The frame is always: I own the design, AI accelerates the typing.

Osilion — the AI recruitment platform

Your most recent and most probed work. Expect 15+ minutes here.

FROM YOUR CV – 'Owned the AI recruitment product end to end'

05 Walk me through that platform — architecture, not features.

WHY THEY ASK

The core system-design question of your interview. They want to see whether you were the owner or a passenger.

YOUR ANSWER

The product automates early-stage recruitment. A recruiter posts a job, candidates apply and upload CVs, the system parses and stores them, an AI conducts a structured first interview, and the platform scores and returns feedback. Architecturally it's a React/Next.js frontend against an ASP.NET Core Web API, with EF Core over Azure SQL. The LLM work — CV scoring in particular — runs as a separate Python service, so the model-facing code stays isolated from the transactional backend and can change without redeploying the API. Slow work runs in the background rather than in a request. The backend is layered: domain rules in the centre, application services orchestrating use cases, infrastructure holding EF Core and the external API clients, and the Web API on the outside. Secrets live in Azure Key Vault and deployment runs through GitHub Actions.

FOLLOW-UPS TO EXPECT

Draw the data model.

Core entities: Job, Candidate, Application, CV/Document, Interview, InterviewQuestion/Response, Score, Feedback. Application joins Candidate and Job and carries workflow state. Scores hang off the Application with the rubric version, so you can re-score without losing history.

Where would this break first under load?

The LLM calls — slowest and most rate-limited, so the queue backs up there first. Then database connections during bulk CV imports. The API itself is least likely, because the heavy work is already off the request path.

What would you change if you rebuilt it?

Define the scoring contract between the C# API and the Python service more strictly from day one — a versioned schema — because that boundary was where most integration friction lived.

TRAP

Have a diagram ready on paper. If you only describe features, you sound like a user of the system rather than its architect.

FROM YOUR CV – 'AI-conducted interviews, scoring, and feedback' + Python LLM service

06 How did you make an unreliable model behave predictably in production?

WHY THEY ASK

This separates people who called an API from people who engineered around one. It is your strongest differentiator in 2026.

YOUR ANSWER

The hard part wasn't calling the model — it was making a non-deterministic component safe inside a transactional system. I kept it in a separate Python service so the boundary was explicit: the C# API sends a scoring request, the Python service owns prompts, model calls and parsing, and returns a validated result. Three things mattered. First, structured output: I asked for JSON against a defined schema and validated it on the way back, rather than parsing prose with regex; if validation failed the job retried and eventually flagged for human review instead of writing a bad score. Second, everything ran as a background job, because model calls take seconds and fail intermittently — so retries with backoff, and the job written to be safe to run twice. Third, grounding: scoring was against a defined rubric with the candidate's actual answers in context, not a free-form 'rate this person', which reduced drift between runs.

FOLLOW-UPS TO EXPECT

Why a separate Python service rather than calling the model from C#?

Two reasons. The Python ecosystem for model work is better — libraries, parsing, iteration speed — and isolating it meant I could change prompts and scoring logic without touching or redeploying the transactional API. It also kept the failure domain separate: if scoring degraded, the rest of the platform kept working.

How did you evaluate whether a prompt change made things better?

Be honest about the rigour you had. If it was a held-out set of CVs with known expected outcomes, say so. If it was systematic spot-checking against known-good cases before rollout, say that — inventing a formal eval harness you didn't build is a trap.

What did you do about hallucinated content?

Grounding and validation. Scores came from a fixed rubric with the actual answers in context, and outputs were schema-validated — anything outside the allowed shape was rejected rather than stored. Anything affecting a hiring decision produced input to a human, not a verdict.

How did you control cost?

Token accounting per request, tight prompts rather than stuffing whole documents, chunking long CVs, not re-scoring unchanged content, and cheaper models for mechanical extraction with the stronger one only where judgement was needed.

TRAP

Don't overclaim on evaluation rigour. If you didn't have a formal eval set, say what you did do — spot-checking against known cases is a legitimate answer.

07 How did CV processing and scoring actually work, end to end?

WHY THEY ASK

A concrete pipeline question. They want detail, not a summary.

YOUR ANSWER

Upload goes to cloud storage and a background job picks it up. Extract text, normalise it, then the Python service pulls structured fields — contact details, experience, skills — from the model against a schema, and separately scores the CV against the job's criteria. The parsed result and the score are stored alongside the raw file, so we never lose the original and a scoring improvement can be re-run over history. Failures are the interesting part: image-only PDFs, unusual layouts, multi-column CVs that extract in the wrong reading order. Those get flagged rather than silently stored wrong.

FOLLOW-UPS TO EXPECT

What formats gave you trouble?

Scanned or image-only PDFs with no text layer, multi-column layouts that extract in the wrong reading order, and CVs built in design tools where text order doesn't match visual order. The mitigation is detecting nonsensical extraction and flagging rather than scoring garbage.

How did you handle a CV that failed to parse?

It went to a failed state with the reason recorded and surfaced for human review, rather than silently producing a low score. Failing loudly matters — a silent zero looks like a real result.

Any privacy considerations with CV data?

Yes — CVs are personal data. Store only what the process needs, control access to candidate records, don't send more than necessary to a third-party model, and be able to delete a candidate's data on request. Worth raising before they ask.

TRAP

Expect a follow-up on personal data. Have a sentence ready about storing only what's needed and controlling access to candidate records.

08 Why Specification? Wasn't that over-engineering?

WHY THEY ASK

A direct challenge to a named pattern. They're testing whether you chose it or copied it.

YOUR ANSWER

It earned its place because candidate filtering was combined in many different ways across use cases — status, role, score threshold, date applied, interview stage — and those combinations kept changing. Without specifications the repository grows twenty near-identical methods that can't be tested independently. With them, each rule is an object I can compose and unit test on its own. I wouldn't reach for it in a simple CRUD app; there the indirection would cost more than it returns.

FOLLOW-UPS TO EXPECT

When would you not use Repository at all?

In Laravel, most of the time — Eloquent is already an abstraction over the database, so wrapping it usually adds indirection without benefit. I'd use one when I genuinely need to swap the data source, or in strict DDD.

Did the layering ever slow you down?

Yes, on simple CRUD features where a change touched four files instead of one. That's the honest cost. It paid off on the parts with real rules — scoring, interview configuration, workflow state — and I'd make the same trade there, but not uniformly.

TRAP

Never defend a pattern on principle. Defend it on the specific problem it solved, and be willing to say where you wouldn't use it.

FROM YOUR CV – 'One of four products delivered in twelve months on a small engineering team'

09 Four products in a year? How deep was your involvement in each?

WHY THEY ASK

A fair challenge. Vagueness here reads as inflation.

YOUR ANSWER

Two I owned end to end — the AI recruitment platform and the lead-generation platform. On the sales CRM I contributed backend features rather than owning the product, and on a 3D container-design tool my involvement was smaller still. It was a small team shipping multiple products for one client, so scope per product was tighter than it would be at a larger company. I'd rather be precise about that than claim four products end to end.

FOLLOW-UPS TO EXPECT

Which one are you proudest of?

The recruitment platform — the one I owned from data model to deployment, and the LLM scoring problem was genuinely hard: making a non-deterministic component behave predictably inside a system that has to be correct.

How did you manage context switching?

Batching by product where possible, written notes on decisions so I could reload context quickly, and being clear with the founders about what would slip when priorities changed. Context switching has a real cost and pretending otherwise produces missed estimates.

TRAP

Do not let this sound like four full products were yours. The honest split is more impressive than the inflated version, and it survives a reference check.

Osilion — lead generation, infrastructure

Fewer questions, but they go straight to engineering judgement.

FROM YOUR CV – 'sending ran off the request cycle in jobs that were safe to retry'

10 What does 'safe to retry' mean concretely? How did you make it idempotent?

WHY THEY ASK

You put idempotency on your CV. They will check that you understand it rather than having heard the word.

YOUR ANSWER

A worker can die after doing the work but before acknowledging it, so any job can run twice — that's normal, not exceptional. For email sending, the risk is emailing the same lead twice. So each send is keyed: before sending I check whether that lead-campaign pair has already been marked sent, and I record the send in the same transaction as the state change. Where possible I set a value rather than incrementing one, so a repeat is a no-op rather than a double count. The provider's own retry behaviour matters too, which is why the job returns fast and doesn't hold the request.

FOLLOW-UPS TO EXPECT

What if the email provider succeeds but your database write fails?

That's the dual-write problem and it's the honest hard case — you can't make two systems atomic. Record intent before sending so you can detect the gap, reconcile against the provider's delivery records, and accept at-least-once as the realistic guarantee.

How did you handle bounces?

Bounce and complaint feedback marks the address so it isn't retried — hard bounces suppressed permanently, soft bounces retried a limited number of times. Ignoring this destroys sender reputation, which is what actually kills a campaign platform.

What's a dead-letter queue for?

After a job exhausts its retry budget it goes to a dead-letter queue instead of retrying forever or vanishing. It parks the message where a human can inspect it, and stops one poison message consuming worker capacity indefinitely.

TRAP

This is your best chance to sound genuinely senior. Rehearse it until it's fluent — it also transfers directly to any payments question.

FROM YOUR CV – 'deployed on Azure (Azure SQL, Key Vault) and Render with GitHub Actions'

11 Walk me through your deployment pipeline. How do you deploy safely?

WHY THEY ASK

Checking that you've operated software, not just written it.

YOUR ANSWER

The pipeline runs on push: install dependencies, run tests, then deploy on main. Secrets never live in the repo — they're in GitHub secrets and Azure Key Vault, pulled at runtime. On deploy order, the rule I follow is migrations first and only backward-compatible ones, so old code never meets a schema it doesn't understand; then the code; then cache and config; then restart background workers, because they hold the old code in memory until you do. Rollback is redeploying the previous build.

FOLLOW-UPS TO EXPECT

How do you do a zero-downtime column rename?

Expand and contract. Add the new column nullable and deploy. Deploy code writing to both. Backfill in batches. Switch reads. Drop the old column in a later release. Never rename in place while old code is running.

What's your rollback plan if a migration breaks?

That's why migrations must be backward-compatible — if the change is additive, rolling the code back is safe on its own. For destructive changes, backup first and test the down migration, but structure changes so rollback never requires undoing a migration under pressure.

How do you know a deploy succeeded?

Health check endpoint, error rate and latency watched for the first minutes after release, and a look at the logs for new exception types. 'It deployed without failing' isn't the same as 'it's working'.

TRAP

The queue-worker restart detail is a strong signal — don't skip it. The expand/contract migration pattern is the expected answer to the follow-up.

FROM YOUR CV – Skills – 'JWT'

12 How did you handle authentication? Why JWT?

WHY THEY ASK

Auth questions are near-universal, and JWT invites a revocation follow-up.

YOUR ANSWER

The API used JWT with short-lived access tokens. The honest trade-off with JWT is that you can't un-issue a token, so revocation is the hard part — which is why the access token expiry is short and the refresh path is what you actually control. The signing secret lived in Key Vault rather than configuration. On the client side, tokens go in httpOnly cookies rather than localStorage, because localStorage is readable by any XSS.

FOLLOW-UPS TO EXPECT

How would you revoke a compromised token immediately?

You can't revoke the access token itself, so you shorten the window and control the refresh: revoke the refresh token server-side, and keep a denylist of token IDs checked per request if immediate revocation is a hard requirement. That reintroduces a lookup — the cost of true revocation with JWT.

Sanctum vs JWT — when would you pick each?

Sanctum for a first-party Laravel app — cookie auth for the SPA, simple revocable tokens for mobile, no OAuth ceremony. JWT when the token must be verified by multiple services without calling a central session store. If I don't need that, I don't take on JWT's revocation problem.

What's in a JWT payload?

Header, payload and signature, base64url-encoded. The payload is signed, not encrypted — anyone can read it, so no secrets go in it. You verify the signature and check expiry, issuer and audience, and reject 'alg: none'.

TRAP

Never say a JWT is encrypted. The payload is base64, readable by anyone — say so before they catch it.

DevVoyage & L2N — the PHP years

The core of the PHP/Laravel CV, and where a Laravel employer will spend most of their time.

FROM YOUR CV – 'booking flows covering availability, pricing, reservation, and checkout'

13 How did you prevent double bookings?

WHY THEY ASK

The classic domain question for anyone with booking systems on their CV. There is no way to bluff it.

YOUR ANSWER

Concurrency on the slot is the whole problem — two requests can both read 'available' before either writes. Three ways to handle it. Pessimistic locking: SELECT ... FOR UPDATE on the slot row inside a transaction, so the second request blocks until the first commits. A unique constraint on the resource-and-slot pair, so the database itself rejects the duplicate. Or optimistic locking with a version column and a retry on conflict. I lean on the database constraint wherever the model allows it, because it still holds even if a future developer forgets the application logic.

FOLLOW-UPS TO EXPECT

What happens when the payment succeeds but the slot was taken?

That's why the slot is held before payment rather than after — a reservation with a short expiry, so the seat is yours while you pay. If it still happens, the resolution is an automatic refund and a clear message, not silently keeping the money.

How do you handle holds that expire?

The hold carries an expiry timestamp and availability queries exclude only unexpired holds — so expiry is effectively automatic at read time rather than depending on a cleanup job running on time. A scheduled job tidies the rows afterwards.

What isolation level were you running?

MySQL/InnoDB defaults to Repeatable Read. That prevents non-repeatable reads within the transaction but does not by itself stop two transactions from both booking — which is exactly why you need the explicit lock or the unique constraint.

TRAP

Don't answer only 'I used a transaction'. A transaction alone does not prevent this — the lock or the constraint does.

FROM YOUR CV – 'integrated payment gateways... for booking and checkout'

14 Walk me through a payment integration. What goes wrong?

WHY THEY ASK

They want to know whether you've handled money in production or just called an SDK.

YOUR ANSWER

The flow is: create the order in a pending state, hand off to the gateway, then confirm from a verified server-side signal — never from the client redirect, because the user can close the tab or the redirect can be forged. So the webhook is the source of truth: verify the signature first, then deduplicate on the provider's event ID because they retry aggressively, then process and return 2xx quickly with heavy work queued. Money is stored as integer minor units or decimal, never float. And the order stores the price at the time of purchase, because prices change and historical orders must not.

FOLLOW-UPS TO EXPECT

What if the webhook arrives before your redirect handler?

It regularly does, and the system has to be fine with it. Both paths converge on the same idempotent state transition keyed on the payment reference — whichever arrives first moves the order to paid, the second is a no-op. If order matters anywhere, that's a design bug.

How do you reconcile with the provider?

A scheduled job compares your records against the provider's for a period and reports discrepancies. They will diverge eventually — a missed webhook, a manual refund in their dashboard — and reconciliation is how you find out before your customer does.

How do you handle partial refunds?

The refund is its own record against the original payment rather than an edit to it, so history stays append-only and the net position is derivable. Never mutate or delete the original payment row.

TRAP

'Never trust the client redirect for payment confirmation' is the sentence that proves you've shipped payments. Say it explicitly.

15 Give me a specific query you optimised and how.

WHY THEY ASK

A specific claim invites a specific example. A generic answer here undermines the bullet.

YOUR ANSWER

The recurring pattern was availability and booking listings that filtered on several columns and sorted by date. Two things dominated. First, N+1 — a listing looping over bookings and lazily loading customer or property on each, which I'd find in the query log or with Debugbar and fix by eager loading. Second, missing or wrongly ordered composite indexes: an index has to match the filter order, so an index on the tenant or property column first, then status, then date, with the range condition last. I'd confirm with EXPLAIN that it wasn't doing a full scan or a filesort rather than assuming.

FOLLOW-UPS TO EXPECT

What does EXPLAIN tell you?

Which index the optimiser chose, how many rows it expects to examine, and whether it's doing a full table scan, filesort or temporary table. The red flag is rows examined vastly exceeding rows returned. EXPLAIN ANALYZE gives actual timings.

When does an index not help?

Low-cardinality columns like booleans, leading-wildcard LIKE '%term', a function wrapped around the column such as YEAR(created_at), implicit type casts, and very small tables where a scan is cheaper anyway.

Why does OFFSET pagination get slow?

The database reads and discards every skipped row — page 1000 means reading 20,000 rows to return 20. Keyset pagination filters on 'id greater than the last seen value' and uses the index directly, so depth stops mattering.

TRAP

Have EXPLAIN vocabulary ready — full table scan, filesort, rows examined. If you can't read a plan aloud, this bullet becomes a liability.

16 How do you safely refactor code with no tests?

WHY THEY ASK

Most companies are hiring you for existing code. This question is really about risk judgement.

YOUR ANSWER

You don't refactor code you can't verify, so the first step is characterisation tests — tests that capture the current behaviour, bugs included, so you know the moment you change it. Then small steps, always green, and never changing behaviour and structure in the same commit. For anything large I prefer the strangler approach: build the new path alongside the old, route part of the traffic to it, expand, then delete the old one — rather than a big-bang rewrite that leaves the system broken for weeks.

FOLLOW-UPS TO EXPECT

How do you decide what's worth refactoring?

Where change is frequent and painful. Ugly code nobody touches costs nothing; moderately messy code in the path of every new feature costs constantly. I follow change frequency, not aesthetic preference.

How do you sell a refactor to a manager?

In delivery terms: 'this coupling means a pricing change takes three days instead of three hours, and we've made two production mistakes because of it'. Then propose it incrementally alongside feature work, not as a project that pauses delivery.

TRAP

For the manager follow-up, translate into delivery cost: 'this coupling means a pricing change takes three days instead of three hours'. Saying 'the code is ugly' loses the argument.

FROM YOUR CV – L2N – 'Five-year tenure'

17 Five years in one place — what changed for you over that time?

WHY THEY ASK

They're probing for growth versus stagnation.

YOUR ANSWER

I came in doing feature work and left owning delivery on client systems end to end — schema, APIs, frontend, deployment — and mentoring juniors through code review. The technical shift was from writing whatever worked to caring about what happens after handover: how it's tested, how it's deployed, how the next person reads it. It's also where most of my legacy modernisation experience comes from, because we were maintaining systems we'd inherited.

FOLLOW-UPS TO EXPECT

Why did you leave?

I'd learned what that environment could teach me and wanted international remote work with more ownership over architecture — which is what DevVoyage, and then the Osilion contract, gave me.

What would you have done differently?

Introduced testing earlier. We added it partway through, and every module written before that was harder to change safely for years afterwards. I now treat tests as part of shipping rather than something to retrofit.

TRAP

Five years is a strength — stability is rare. Don't be apologetic about it.

CoreBari — the side product

Impressive technically, but you must control how it's framed.

FROM YOUR CV – 'CoreBari – independent SaaS product, live in production'

18 Tell me about CoreBari. Is it a business? Will it distract you?

WHY THEY ASK

The concern is divided attention and flight risk. Answer the technical part with enthusiasm and the commitment part briefly and calmly.

YOUR ANSWER

It's a SaaS product I built for small businesses — point of sale, diagnostics lab, attendance and payroll, invoicing — sharing one identity layer so a customer created in one app is visible in another. It mostly runs itself day to day. For me it's where I stay hands-on with the harder problems: multi-tenancy, offline-first sync, payment integration. My priority is a stable long-term role, and honestly the product work makes me a better engineer for it.

FOLLOW-UPS TO EXPECT

How much time does it take?

Not much ongoing — the build was the work and it's stable now. Evenings and weekends when I want to add something. It has never affected client delivery, and it wouldn't.

What happens if it grows?

It would still be a product, not a job. If it ever reached a scale demanding full-time attention I'd have that conversation openly rather than quietly.

Do you have customers?

Answer factually — however many businesses actually use it. Never inflate: it's checkable, and 'a small number of real users' is more credible than a vague large claim.

TRAP

Never use the words founder, owner, or 'I run a company'. 'A product I built' is accurate and carries none of the risk.

FROM YOUR CV – 'offline-first PWA synchronisation'

19 How does the offline sync work? How do you resolve conflicts?

WHY THEY ASK

The most technically impressive thing on your CV, and very few candidates can discuss it. Expect them to dig.

YOUR ANSWER

The requirement is that a shop keeps selling when the internet drops, so the device has to be authoritative during an outage. Writes go to local storage immediately and queue in an outbox, so the UI never blocks on the network. A background process drains the outbox when connectivity returns and the server responds with authoritative state. Every operation carries a client-generated ID so a replayed sync doesn't duplicate a sale. On conflicts — the best answer is to model them away: a sale is append-only, so two devices creating sales don't actually conflict, they both happened. Where genuine divergence is possible I go server-authoritative per field. And device clocks lie, sometimes by hours, so anything authoritative uses server time rather than the device's.

FOLLOW-UPS TO EXPECT

What happens if the same device syncs twice?

Nothing, by design. Each operation carries a client-generated ID and the server treats a repeat as a no-op, returning the original result. Same idempotency principle as the payment and email work.

How do you migrate the local schema?

The genuinely hard operational problem. The client stores a schema version and runs migrations against local data on upgrade, and the sync protocol has to tolerate a device a version behind — you cannot force every device to update before it next connects.

What if a sync partially fails?

Operations drain individually rather than as one batch, so a failure stops that operation and leaves the rest of the outbox intact and retryable. What you must avoid is marking the whole batch synced when only part succeeded.

TRAP

Practise this until it's six confident minutes with a drawing. It's the single strongest technical story you own.

FROM YOUR CV – 'sharing one identity layer with single sign-on across apps'

20 How does SSO across your apps work?

WHY THEY ASK

A natural follow-up, and multi-tenancy questions usually come with it.

YOUR ANSWER

One identity service issues the session, and each app trusts it rather than holding its own users, so a user signs in once and moves between apps. Entitlements are separate from identity — which apps a tenant has access to comes from their subscription, checked per request. On tenancy, every record is scoped to a tenant and scoping is the default rather than something you opt into, because the top risk in a system like this is one forgotten filter leaking another business's data.

FOLLOW-UPS TO EXPECT

How do you guarantee no cross-tenant leakage?

By making scoping impossible to forget rather than relying on discipline — tenant context resolved once per request and applied by default, so an unscoped query is the exception you must write deliberately. Row-level security in the database is the stronger version.

What are the multi-tenancy options and why did you pick yours?

Shared table with a tenant column — cheapest, but one missed filter leaks data. Row-level security — enforcement in the database, safer. Schema per tenant — better isolation, migrations multiply. Database per tenant — strongest, heaviest. I chose shared with strict scoping because shared data across apps is the point of the suite.

TRAP

Be ready to name the four tenancy models — shared table with tenant ID, row-level security, schema per tenant, database per tenant — and the trade-off you accepted.

FROM YOUR CV — 'mobile-wallet payment integrations'

21 What payment providers have you integrated?

WHY THEY ASK

Straightforward, but this is where you convert local experience into transferable skill.

YOUR ANSWER

Mobile wallet providers in the Bangladesh market for CoreBari, and card gateways for the travel and e-commerce work at DevVoyage. The providers differ, but the engineering is the same everywhere: verify webhook signatures, deduplicate on the provider's event ID because they retry, never confirm from a client redirect, store money as minor units, and reconcile against the provider's records because the two will diverge eventually.

FOLLOW-UPS TO EXPECT

Have you worked with Stripe?

If you haven't, say so plainly and bridge: 'Not Stripe specifically — the gateways I integrated were regional. The model is the same one Stripe documents: intent, webhook confirmation with signature verification, idempotency keys, reconciliation. I'd be productive on it quickly.'

How would you approach an unfamiliar provider?

Read their webhook and idempotency documentation first, because that's where the correctness risk lives — not the happy path. Build against the sandbox, deliberately replay events and simulate failures, then wire it into real order state.

TRAP

If they ask about Stripe specifically and you haven't used it, say so — then describe the pattern above. The pattern is what transfers.

The danger zone

Your genuine weak points. Prepared honest answers beat improvised defensive ones.

FROM YOUR CV — Skills — 'Additionally: C#, ASP.NET Core, EF Core, Azure'

22 How strong is your .NET really? It's only been a year.

WHY THEY ASK

They can read the dates. Answering this honestly and confidently is worth more than any exaggeration.

YOUR ANSWER

One year, but a year of production ownership rather than tutorials — I architected and shipped the recruitment platform's backend on it. What transferred instantly was the architecture: dependency injection, middleware, ORM behaviour, migrations, API design are the same problems I'd been solving in Laravel for years, just different syntax. What I had to learn specifically was LINQ and deferred execution, async/await properly, EF Core's change tracking, and Azure. I'd describe myself as solidly productive in .NET rather than an expert in its internals — my deepest ecosystem is still PHP.

FOLLOW-UPS TO EXPECT

What's deferred execution?

A LINQ query over IQueryable isn't executed when written — it's translated to SQL and runs when you materialise it with ToList, First, Count. The bug is calling ToList too early, which pulls the whole table into memory and filters in C# instead of the database.

Explain DI lifetimes.

Transient is a new instance per resolution, scoped is one per request, singleton is one for the application's lifetime. The classic bug is injecting a scoped DbContext into a singleton — it captures one request's context forever and corrupts everything afterwards.

AsNoTracking — what and why?

EF Core tracks entities to detect changes on save. For read-only queries that tracking is pure overhead and memory, so AsNoTracking skips it. A straightforward win on list and report queries you'll never save.

TRAP

Do not inflate here. .NET interviewers probe internals within two questions, and an exposed bluff costs more than the honest boundary.

23 You haven't written Laravel for a year. Is it still current?

WHY THEY ASK

A fair and likely question. Pre-empting it is better than being caught by it.

YOUR ANSWER

Eight years of it, and the past year on .NET didn't erase it — if anything the architectural work sharpened how I think about the service container and layering. I've been actively back in it: rebuilding APIs with Sanctum, policies, queues and Pest tests to make sure I'm current on PHP 8 features like enums and readonly, and on what's changed in recent Laravel versions.

FOLLOW-UPS TO EXPECT

What's new in PHP 8 that you actually use?

Enums above all — status fields became real types with behaviour instead of loose constants. Readonly properties made DTOs genuinely immutable. Constructor promotion cut boilerplate. And match replaced most switches because it's strict and returns a value.

What changed in the recent Laravel versions?

Be specific about what you've verified rather than guessing version numbers — the streamlined application skeleton and configuration, Pest as the common testing choice, native enum casting on models. If unsure of a version detail, say you'd check rather than inventing it.

TRAP

You must be able to name enums, readonly, match, and constructor promotion instantly. If you can't, do that lesson before this interview.

24 Can you quantify any of this? What was the impact?

WHY THEY ASK

Some interviewers push for numbers. You removed unverifiable ones deliberately — say so.

YOUR ANSWER

I'm careful with numbers I can't stand behind, so I'd rather give you the concrete scope. On the recruitment platform, the whole first-stage interview process — posting, intake, CV parsing, interview, scoring — ran through the system rather than a recruiter's inbox. The booking platforms replaced manual booking and customer management entirely and ran on real production traffic. Where I don't have a measured figure I'd rather describe what changed than invent a percentage.

FOLLOW-UPS TO EXPECT

Weren't you tracking anything?

At the startup, product analytics weren't mature — decisions came from customer feedback rather than dashboards. That's normal at that stage, and I'd rather tell you that than quote a number I couldn't reproduce.

How did you know it was working?

Usage, and the absence of the manual process it replaced — recruiters screening through the platform instead of their inbox, bookings coming through the system rather than by phone. Plus operational signals: error rates, failed jobs, queue depth.

TRAP

This answer scores well with good interviewers. Never invent a number under pressure — a fabricated metric that unravels is fatal.

25 What makes you senior rather than mid-level?

WHY THEY ASK

They're testing self-awareness. Listing years is the weak answer.

YOUR ANSWER

Owning outcomes rather than tickets. I take a vague requirement to a shipped, deployed, maintainable feature — data model, API, interface, tests, deployment — and I make the trade-off calls along the way and explain them. It also means knowing what not to build: I've argued against architecture that would have been fun and wrong for the team's size. And it means the code I leave behind is something the next person can work in, which is most of what mentoring at DevVoyage and L2N was about.

FOLLOW-UPS TO EXPECT

Tell me about a trade-off you got wrong.

Have a real one. A defensible example: applying full layering uniformly including the simple CRUD parts, so small changes touched four files for no benefit. The lesson is matching structure to the complexity of each area.

How do you mentor?

Mostly through code review — explaining why rather than just requesting changes, and separating blocking issues from preferences so juniors know what actually matters. And pairing on the first instance of something new.

TRAP

Have a real example of an architectural decision you argued against. 'I do what I'm told well' is a mid-level answer.

26 When can you start? What are your salary expectations?

WHY THEY ASK

Practical screening. Hesitation here reads as uncertainty.

YOUR ANSWER

I'm available from the start of October, so effectively immediately. On compensation, I'd like to understand the scope and level first — what range is budgeted for the role? [If pressed] Based on the market for a senior full-stack role at this level I'm looking at [your range], and I'm flexible for the right fit.

FOLLOW-UPS TO EXPECT

Are you interviewing elsewhere?

Yes, I'm in a few processes — said neutrally, without naming companies or bluffing about offers you don't have. It's normal and signals demand. Inventing a competing offer can collapse badly.

Contract or permanent preference?

Permanent — I want to stay with one product long enough to see decisions play out. Open to contract if the role is right, but stability is what I'm optimising for now.

TRAP

Decide your target, minimum and walk-away numbers before the first call. Never accept an offer on the call — ask for a day.

Questions you ask them

Never say 'no, I'm good'. Pick three; the last two are the ones that matter most for a remote contract-to-permanent move.

27 Your questions for the interviewer

WHY THEY ASK

This is assessed. Good questions signal seniority and genuine interest.

YOUR ANSWER

How is the team structured, and how do technical decisions actually get made?

What does the codebase look like today — where's the biggest source of technical debt?

What would success in this role look like in the first 90 days?

How does the team handle async work across timezones?

Why is this role open?

What does the deployment and on-call setup look like?

TRAP

Never open with salary, holidays, or working hours. Those come after they want you.